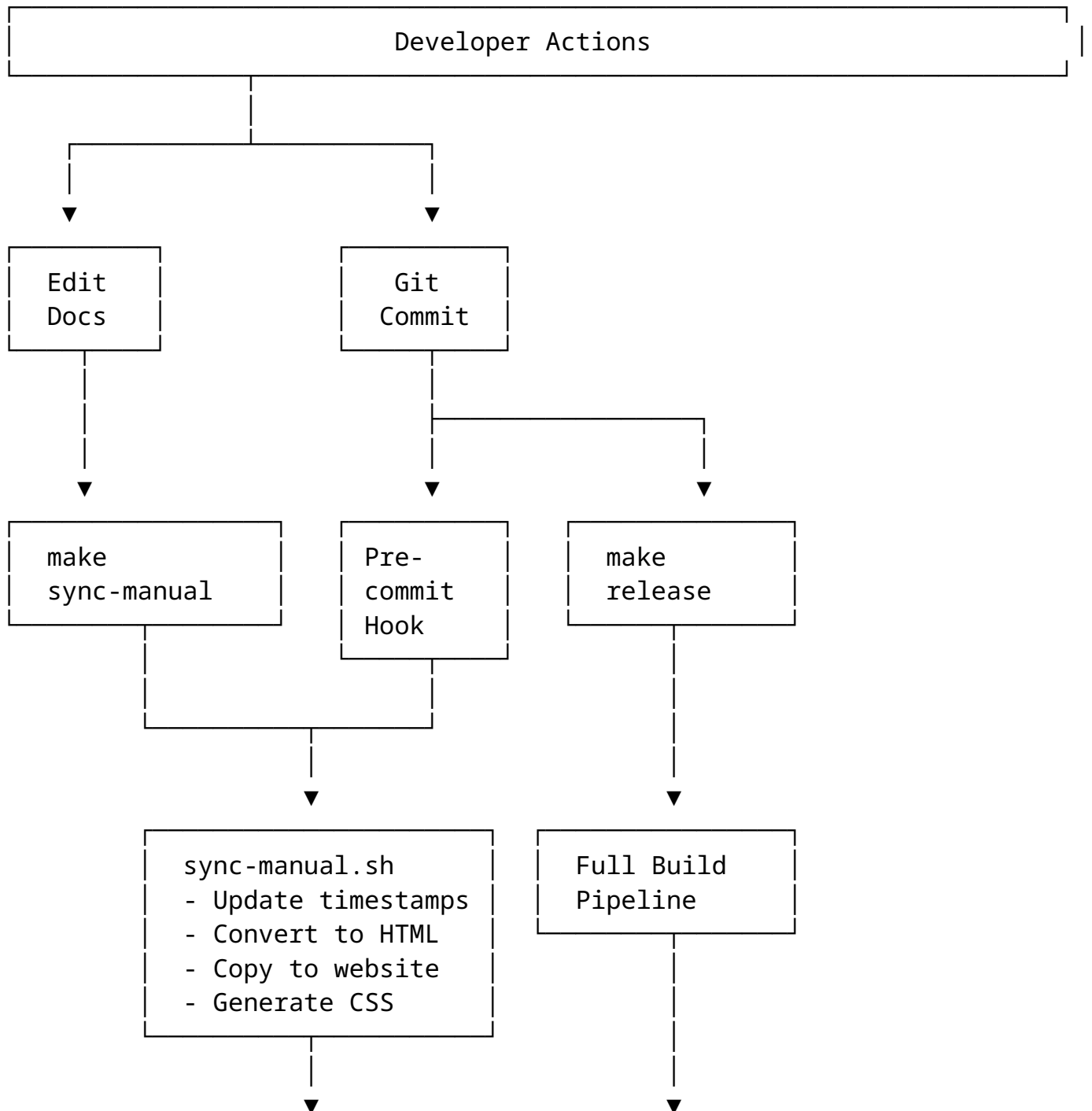
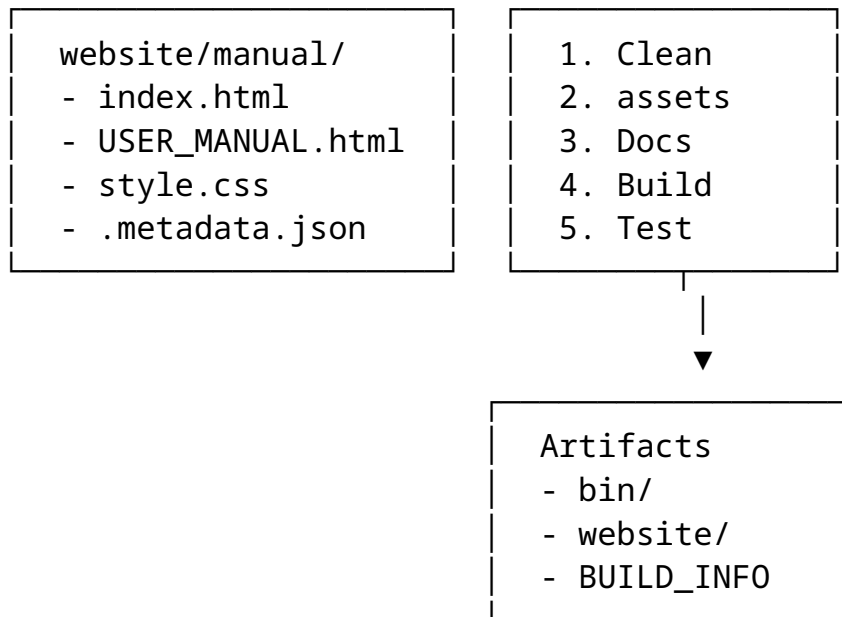


BUILD_WORKFLOW

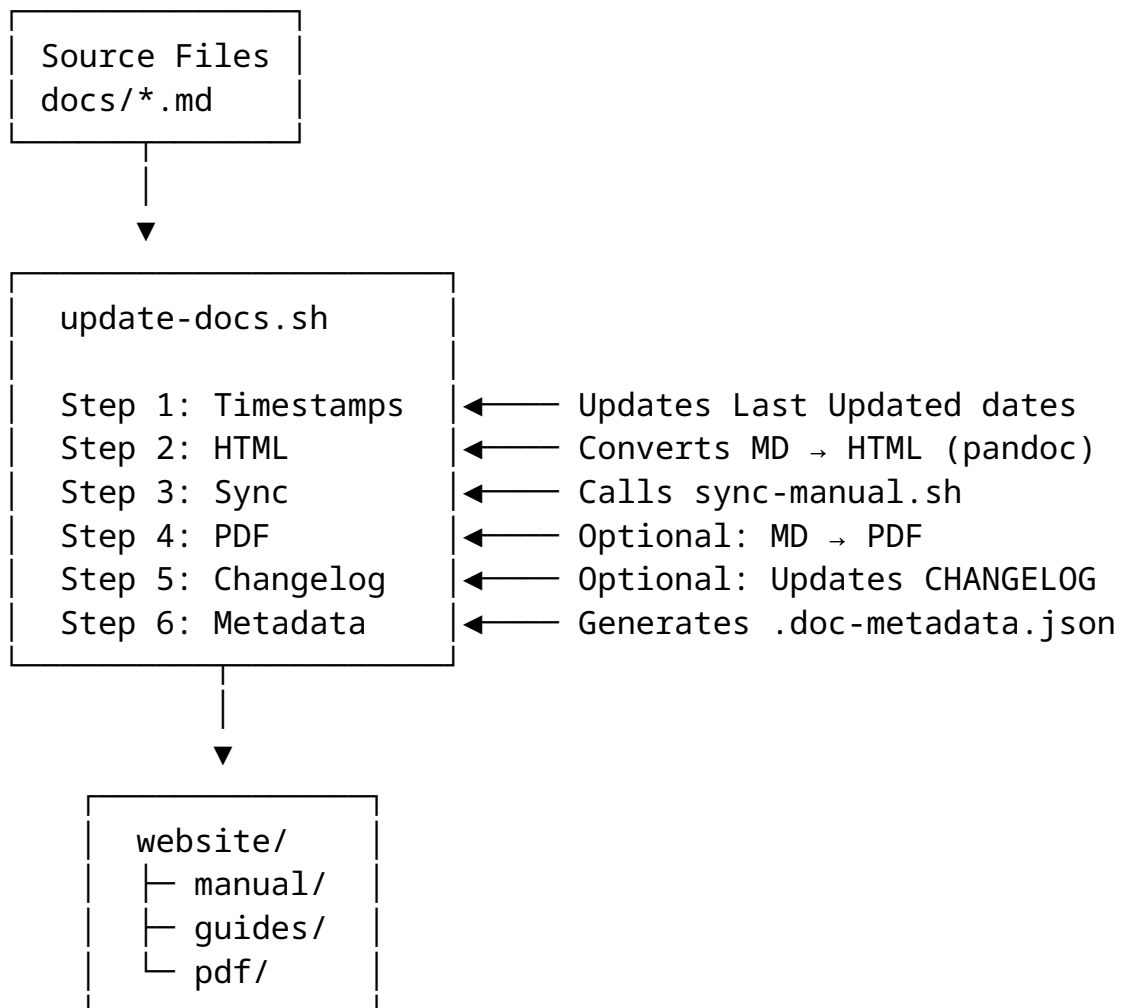
HelixCode Build Workflow Documentation

Complete Build System Architecture

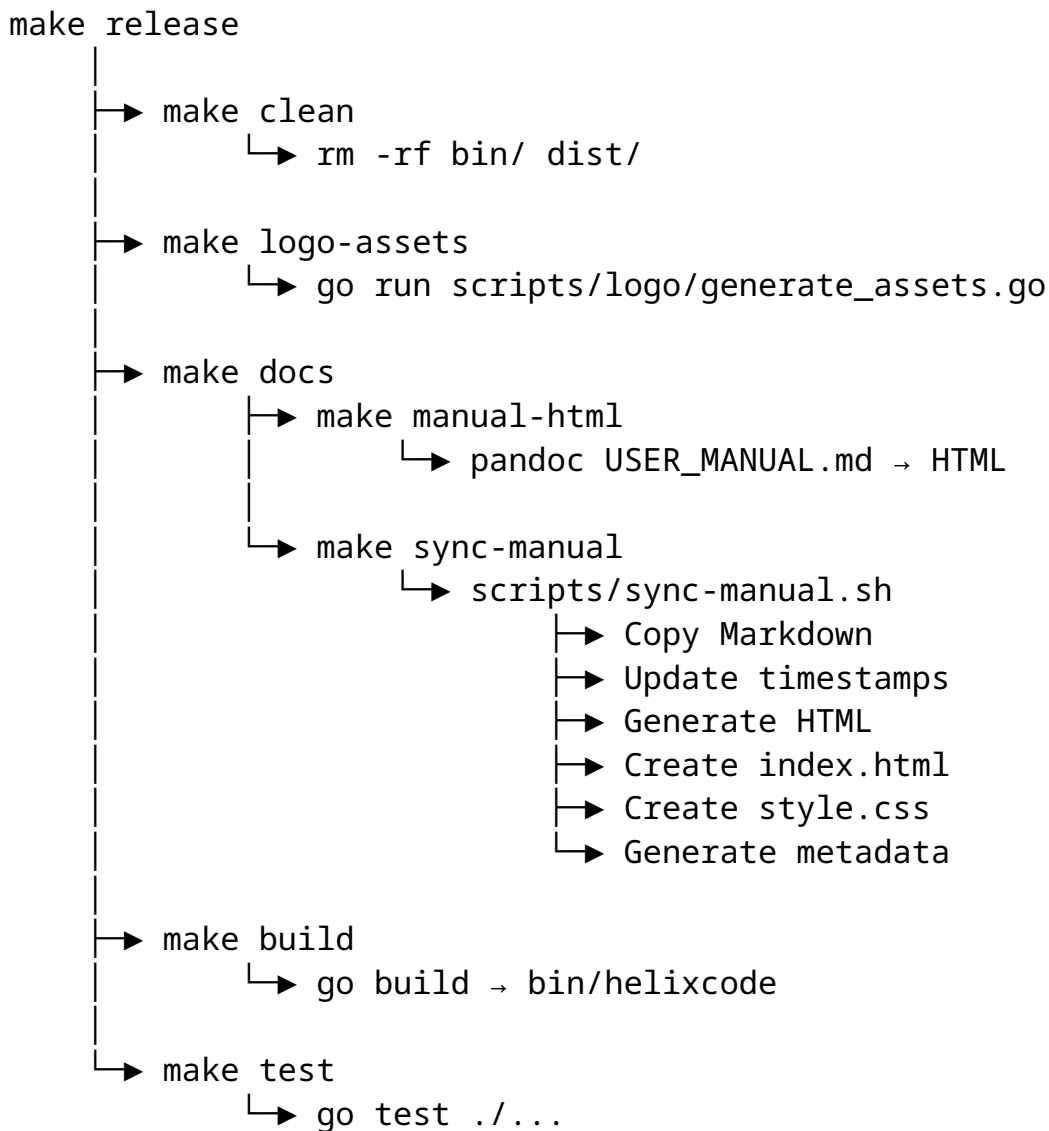




Documentation Update Flow

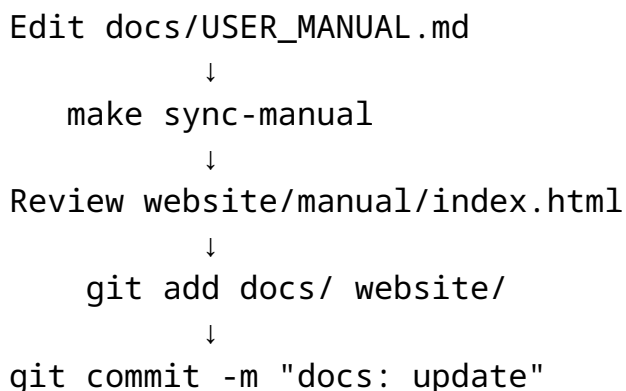


Make Target Dependencies



Developer Workflows

Workflow 1: Quick Documentation Update



↓
git push

Workflow 2: Full Documentation Update

Edit docs/*.md
↓
./scripts/update-docs.sh --pdf --changelog
↓
Review all changes
↓
git add docs/ website/ CHANGELOG.md
↓
git commit -m "docs: comprehensive update"
↓
git push

Workflow 3: Release Build

Update version in Makefile
↓
./scripts/update-docs.sh --pdf --changelog
↓
make release
↓
Verify bin/ and website/
↓
git add .
↓
git commit -m "release: vX.Y.Z"
↓
git tag vX.Y.Z
↓
git push --tags

Workflow 4: With Pre-Commit Hook

Install hook (one-time):
cp scripts/pre-commit.template .git/hooks/pre-commit
↓
Edit docs/USER_MANUAL.md
↓
git add docs/USER_MANUAL.md
↓

```
git commit -m "docs: update"
  ↓
[Hook runs automatically]
  ├──> Updates timestamps
  ├──> Regenerates HTML
  └──> Stages website/
  ↓
Commit proceeds
  ↓
git push
```

Script Execution Flow

sync-manual.sh Execution

```
START
|
|→ Check prerequisites
|   ├──> Does USER_MANUAL.md exist?
|   └──> Is pandoc available?
|
|→ Create directories
|   └──> mkdir -p website/manual
|
|→ Copy Markdown
|   └──> cp docs/USER_MANUAL.md → website/manual/
|
|→ Update timestamp
|   └──> sed "s/Last Updated.../$(date)/"
|
|→ Convert to HTML (if pandoc)
|   └──> pandoc → USER_MANUAL.html
|
|→ Generate index.html
|   └──> Create landing page with links
|
|→ Generate style.css
|   └──> Create stylesheet
|
|→ Generate metadata
|   └──> Create .sync-metadata.json
|
END (Success)
```

update-docs.sh Execution

START

- Parse arguments (--pdf, --changelog, --verbose)
- Step 1/6: Update timestamps
 - ↳ For each Markdown file:
 - ↳ sed "s/Last Updated.../\$(date)/"
- Step 2/6: Convert to HTML
 - ↳ USER_MANUAL.md → HTML
 - ↳ USER_GUIDE.md → HTML
 - ↳ API_REFERENCE.md → HTML
- Step 3/6: Sync to website
 - ↳ Execute sync-manual.sh
- Step 4/6: Generate PDF (optional)
 - ↳ If --pdf flag:
 - ↳ pandoc → HelixCode_User_Manual.pdf
- Step 5/6: Update changelog (optional)
 - ↳ If --changelog flag:
 - ↳ Prepend entry to CHANGELOG.md
- ↳ Step 6/6: Generate metadata
 - ↳ Create .doc-metadata.json

END (Success)

build.sh Execution

START

- Parse arguments
 - ↳ --no-docs
 - ↳ --with-tests
 - ↳ --platform [linux|darwin|windows|all]
- Step 1/6: Check prerequisites
 - ↳ Go installed?
 - ↳ pandoc installed? (if docs enabled)

```

└─ Step 2/6: Generate assets
    └─ go run scripts/logo/generate_assets.go

└─ Step 3/6: Build documentation (if enabled)
    └─ Execute sync-manual.sh

└─ Step 4/6: Build application
    └─ For each platform:
        └─ go build → bin/helixcode-[platform]

└─ Step 5/6: Run tests (if --with-tests)
    └─ go test ./...

└─ Step 6/6: Generate build info
    └─ Create bin/BUILD_INFO.json

|
END (Success)

```

File Generation Timeline

On make sync-manual:

Time	Action	Output
0s	Start script	
1s	Copy Markdown	website/manual/USER_MANUAL.md
2s	Update timestamp	(modified in place)
3s	Convert to HTML	website/manual/USER_MANUAL.html
4s	Generate index	website/manual/index.html
4s	Generate CSS	website/manual/style.css
5s	Generate metadata	website/manual/.sync-metadata.json
5s	Complete	

On ./scripts/update-docs.sh --pdf:

Time	Action	Output
0s	Start script	
1s	Update timestamps	docs/*.md (modified)
2s	Convert USER_MANUAL	website/manual/USER_MANUAL.html
3s	Convert USER_GUIDE	website/guides/USER_GUIDE.html
4s	Convert API_REFERENCE	website/guides/API_REFERENCE.html
5s	Execute sync-manual.sh	(see above timeline)
10s	Generate PDF	website/pdf/HelixCode_User_Manual.pdf

15s	Generate metadata	website/.doc-metadata.json
15s	Complete	

On make release:

Time	Action	Output
0s	make clean	rm -rf bin/ dist/
1s	make logo-assets	assets/logo/*
2s	make docs	website/* (see above)
7s	make build	bin/helixcode*
10s	make test	(test output)
30s	Complete	

Integration Points

1. Makefile Integration

Direct targets

sync-manual → scripts/sync-manual.sh
manual-html → pandoc conversion only
docs → manual-html + sync-manual

Composite targets

assets → logo-assets + sync-manual
release → clean + logo-assets + docs + build + test

2. Git Hook Integration

Pre-commit hook triggers on:

git add docs/*.md
git commit

Hook execution:

```
.git/hooks/pre-commit
├─> Detect changed docs
├─> Update timestamps
├─> Run sync-manual.sh
└─> Stage website/
```

3. CI/CD Integration

GitHub Actions example
- name: Build Docs

run: make docs

- name: Build App

run: make build

- name: Release

run: make release

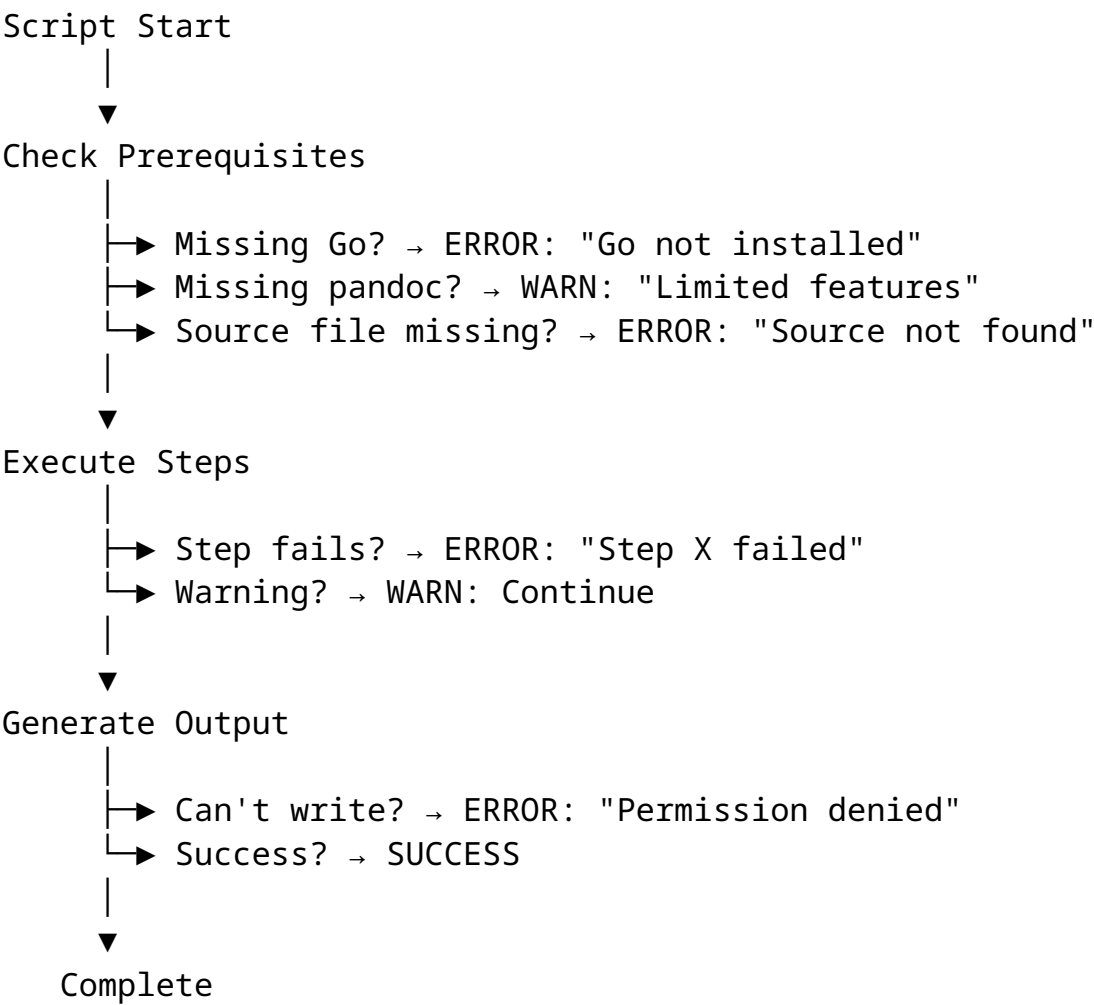
Command Comparison

Task	Make Command	Script Command	Pre-commit Hook
Quick sync	make sync-manual	./scripts/sync-manual.sh	Auto on commit
HTML only	make manual-html	N/A	N/A
Full docs	make docs	./scripts/update-docs.sh	N/A
With PDF	N/A	./scripts/update-docs.sh --pdf	N/A
Release	make release	./scripts/build.sh --with-tests	N/A

Output Directory Structure

```
website/
├── manual/
│   ├── index.html           ← Landing page
│   ├── USER_MANUAL.html    ← HTML version
│   ├── USER_MANUAL.md      ← Markdown copy
│   ├── style.css            ← Stylesheet
│   └── .sync-metadata.json  ← Sync info
├── guides/
│   ├── USER_GUIDE.html
│   └── API_REFERENCE.html
├── pdf/
│   └── HelixCode_User_Manual.pdf
└── .doc-metadata.json      ← Global metadata
```

Error Handling Flow



Performance Metrics

Typical Execution Times

Operation	Time	Notes
make sync-manual	5s	With pandoc
make docs	15s	All documentation
./scripts/update-docs.sh	15s	Without PDF
./scripts/update-docs.sh --pdf	45s	With PDF (requires LaTeX)
make build	30s	Single platform
make release	60s	Full release with tests

File Sizes

File	Size	Notes
USER_MANUAL.md	~50KB	Source Markdown
USER_MANUAL.html	~75KB	HTML with CSS
USER_MANUAL.pdf	~150KB	PDF version
style.css	~3KB	Stylesheet
index.html	~2KB	Landing page

Version: 1.0.0 **Last Updated:** 2025-11-06 **Status:** Production Ready